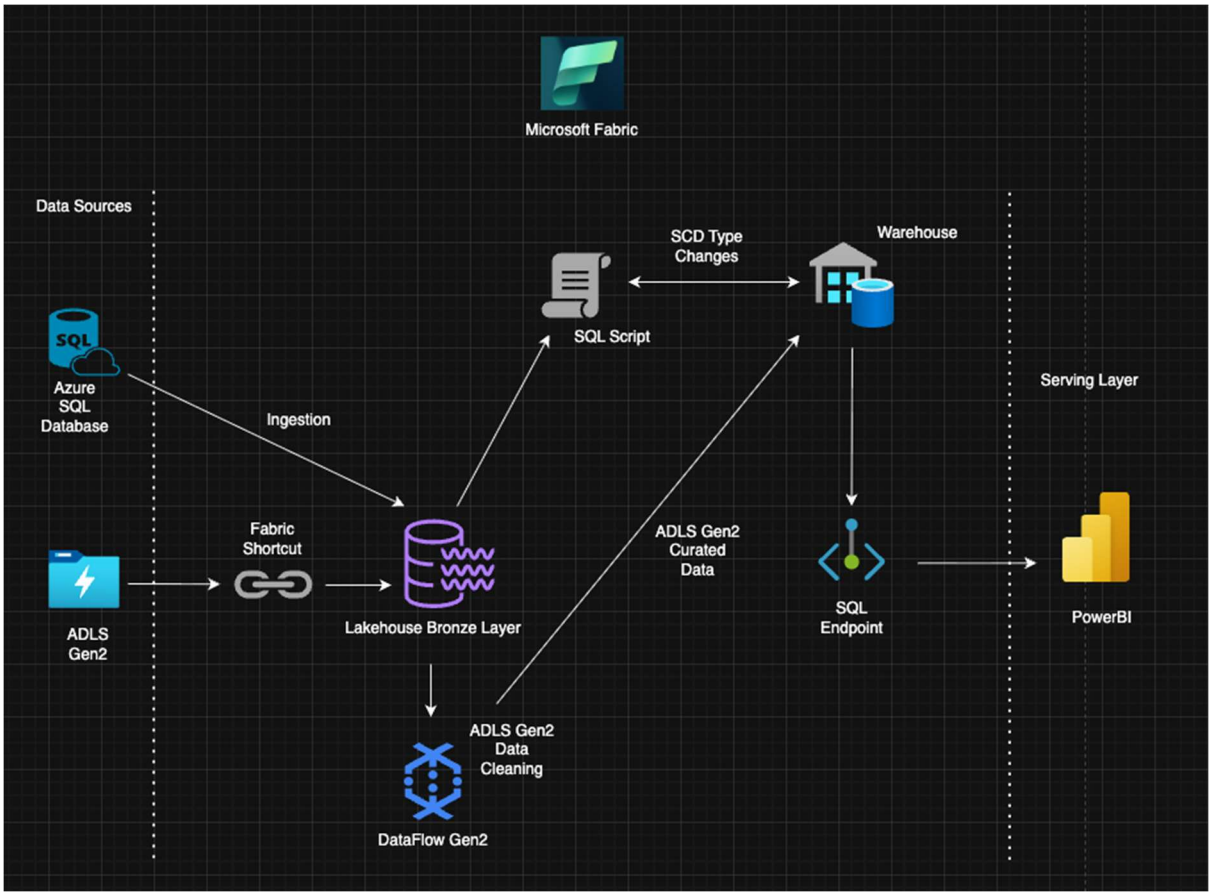
Microsoft Fabric ETL Project
Name: Vemula Raghunath Goud
Project: End-to-End Data Pipeline using Microsoft Fabric (Lakehouse + Dataflow Gen2 + Warehouse)

Contents

Architecture Overview 2
1.Project Overview: 2
2. Technologies Used 3
3. Data Sources 3
4. Data Ingestion Strategy 4
5. Data Transformation (Dataflow Gen2) 9
6. Implementation Method 18
7. Semantic Model & Reporting: 20
8. Challenges Faced 22
9. Learnings & Enhancements 22
10. Summary 23



Architecture Overview

Azure SQL DB ADLS Gen2
↓ ↓
Invoke Pipeline Shortcut (Lakehouse Files)
↓ ↓
azureSQLdbIngestion Dataflow Gen2 (Transform)
↓ ↓
Fabric Warehouse (Tables)
↓
Stored Procedures (SCD1, SCD2)
↓
Semantic Model / Reporting

1.Project Overview:

Objective:
To design and implement a scalable data pipeline that ingests data from multiple sources (Azure SQL DB and ADLS Gen2), performs transformations using Dataflow Gen2, and loads curated data into a Fabric Warehouse with Slowly Changing Dimension (SCD) handling.

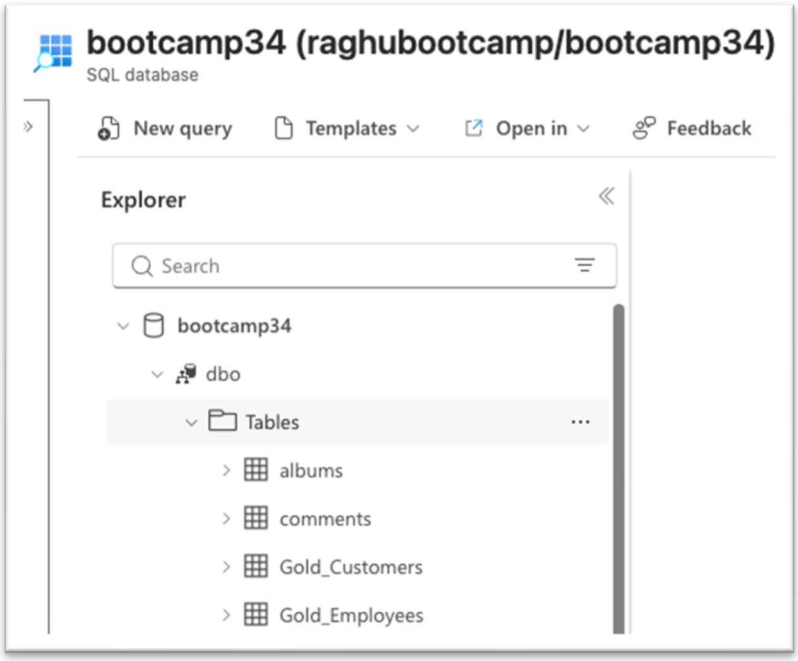
2. Technologies Used

Layer	Technology
Source 1	Azure SQL Database Tables
Source 2	ADLS Gen2 Files
Ingestion	Fabric Pipeline (Invoke Pipeline - Legacy)
Storage	Fabric Lakehouse (Shortcut + Files)
Transformation	Dataflow Gen2
Data Warehouse	Fabric Warehouse
Processing Logic	Stored Procedures (SCD1, SCD2)
Orchestration	Fabric Pipelines
Reporting	Semantic Model (Power BI)

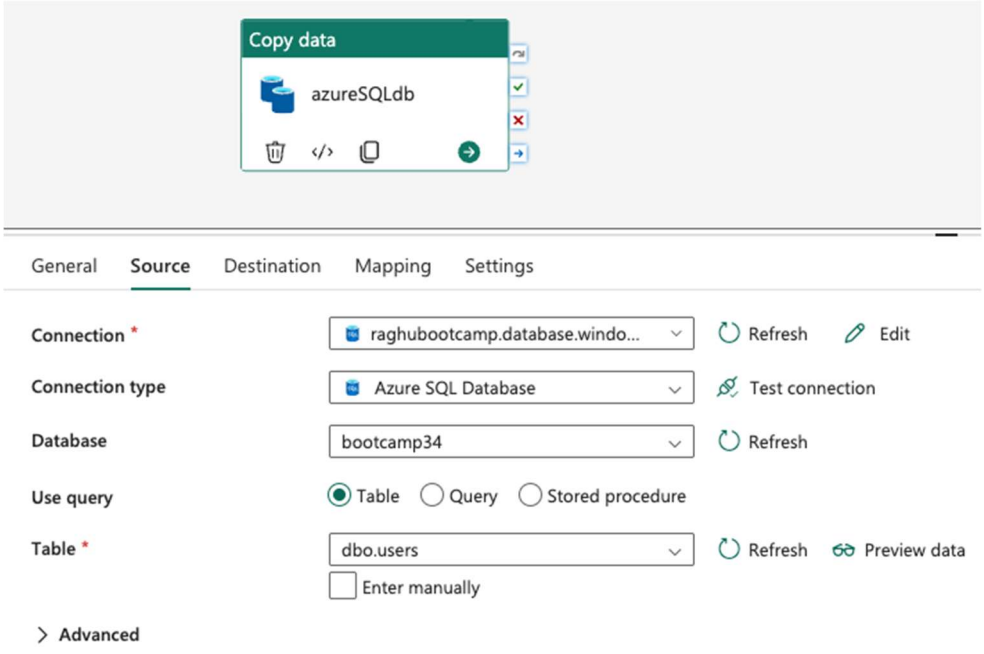
3. Data Sources

1. Azure SQL DB

- Contains structured relational data tables in database
- Ingested using pipeline activity

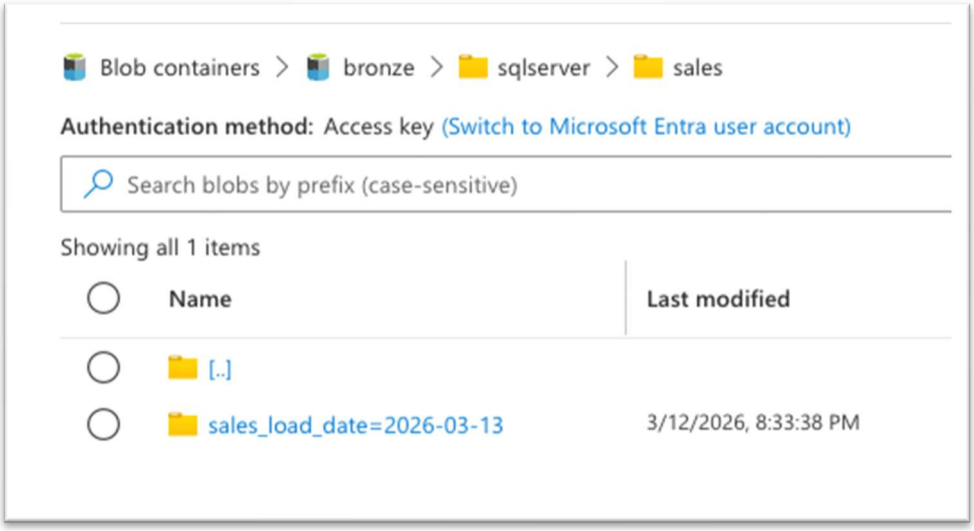


Creating a pipeline using COPY data activity with Azure SQL Database as source and Destination set to Fabric Lakehouse.



2. ADLS Gen2

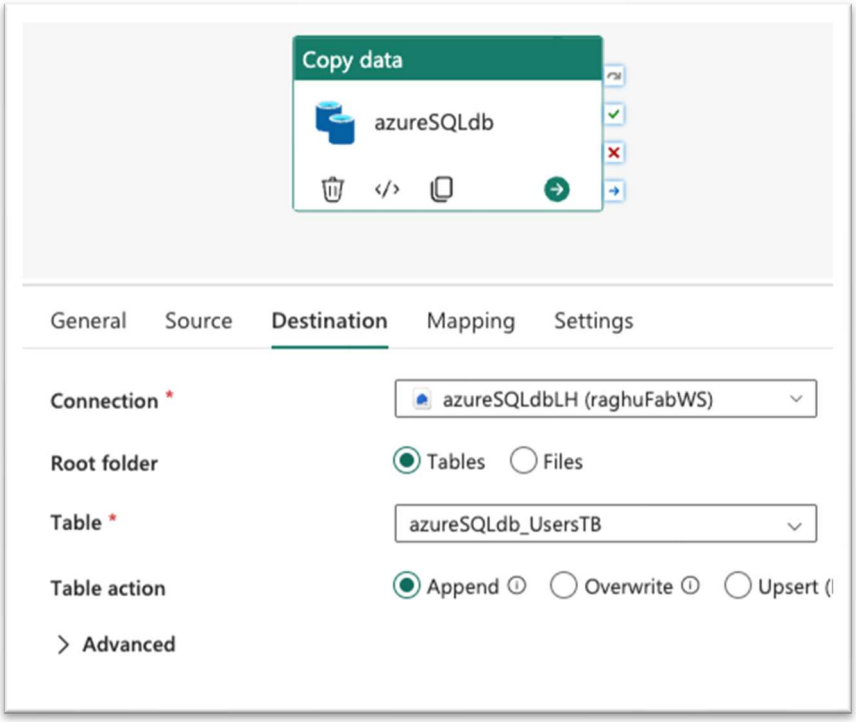
- File-based data (CSV/Parquet)
- Accessed using **Shortcut in Lakehouse**



4. Data Ingestion Strategy

1. Azure SQL DB
Invoked using: **Invoke Pipeline (Legacy)**

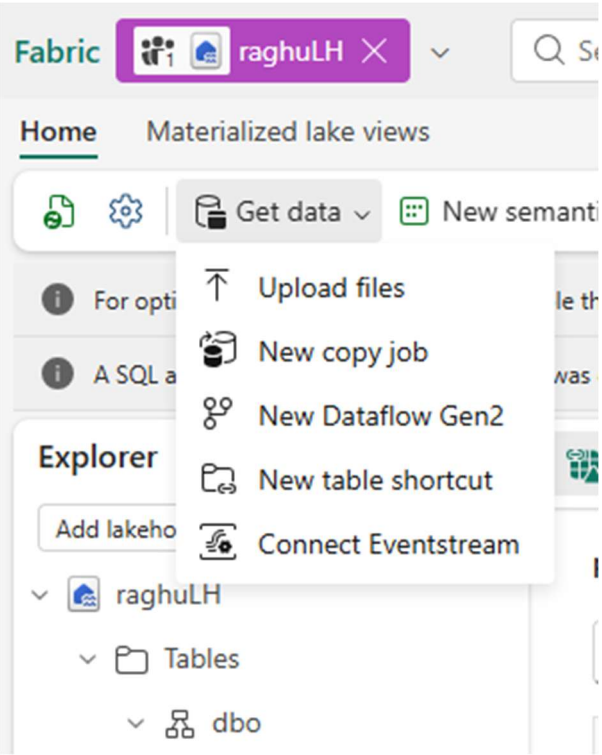
- Loads structured data into Warehouse staging tables



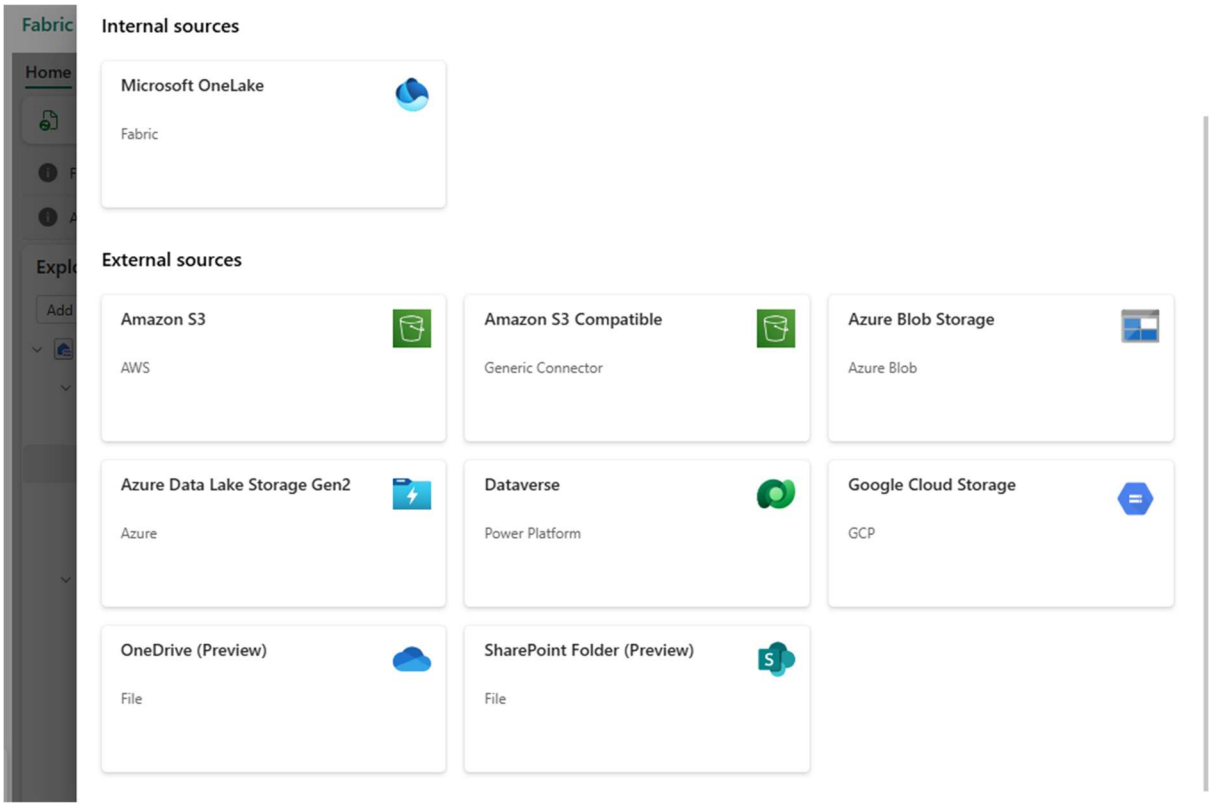
2. ADLS Gen2

- Integrated using: **Shortcut in Lakehouse (Files section)**

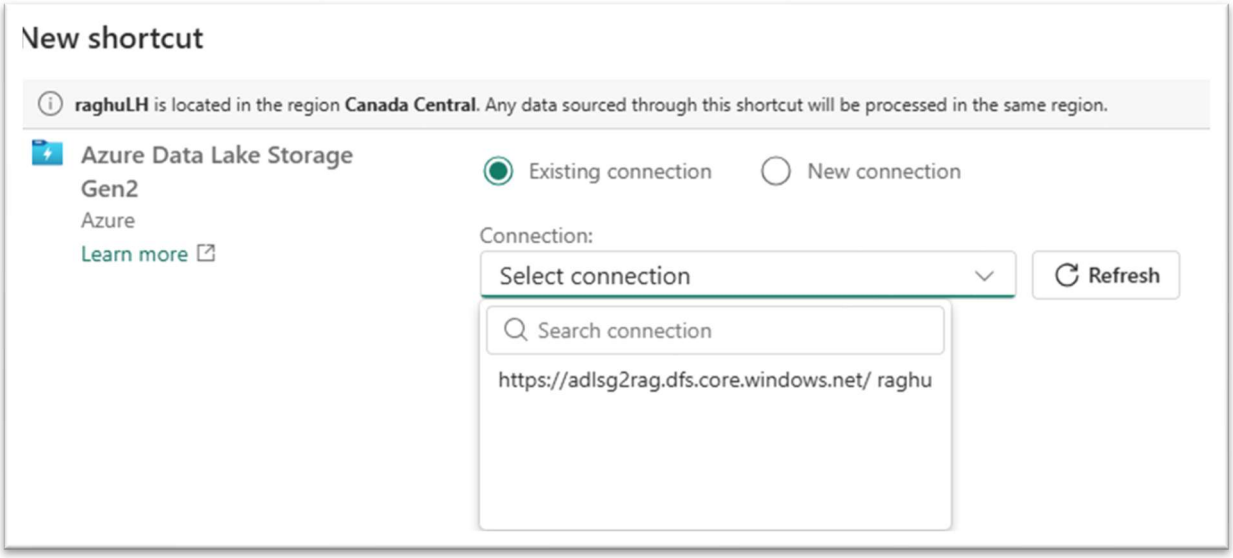
Shortcut option under get data tab.



Picking up source option for Shortcut: ADLS Gen2

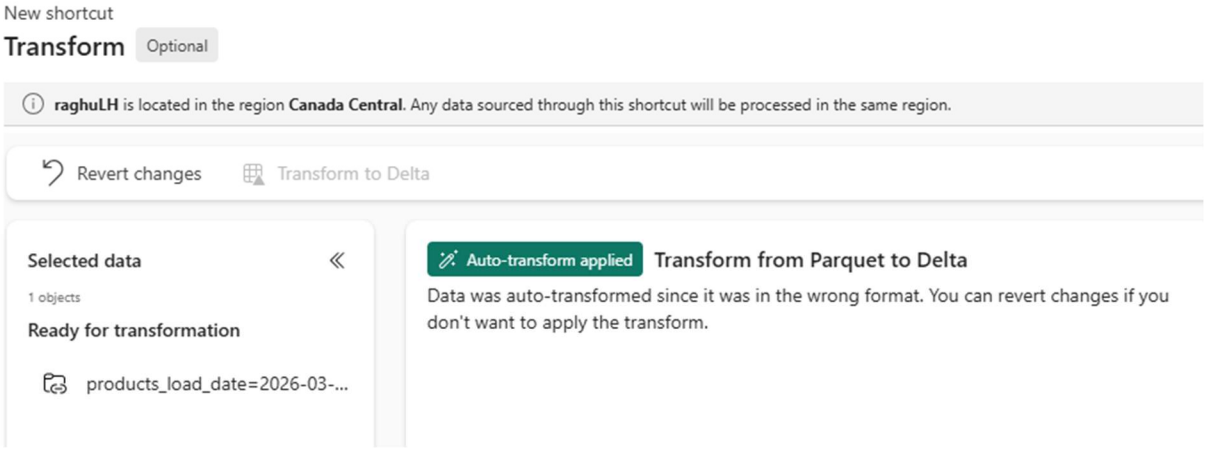


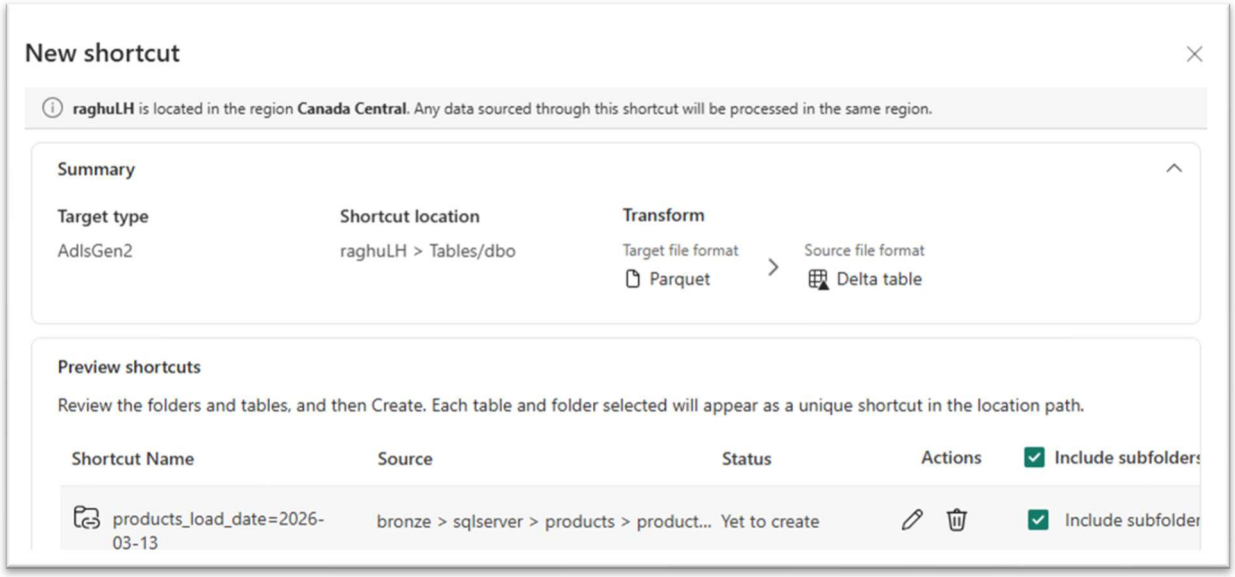
For ADLS Gen2 storge account; use the Resource URL for connection



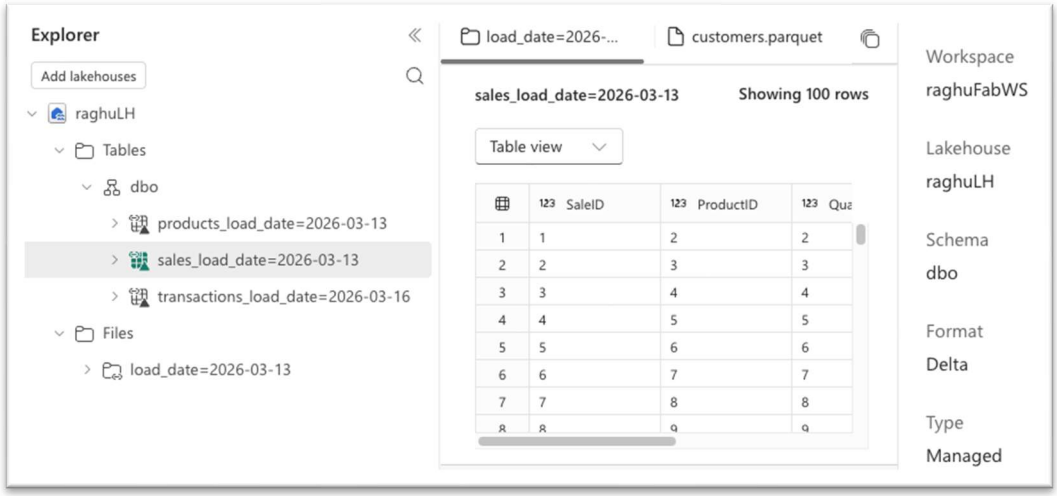
Pick required data file/table source.

Data to be ingested automatically gets transformed to Delta format table.

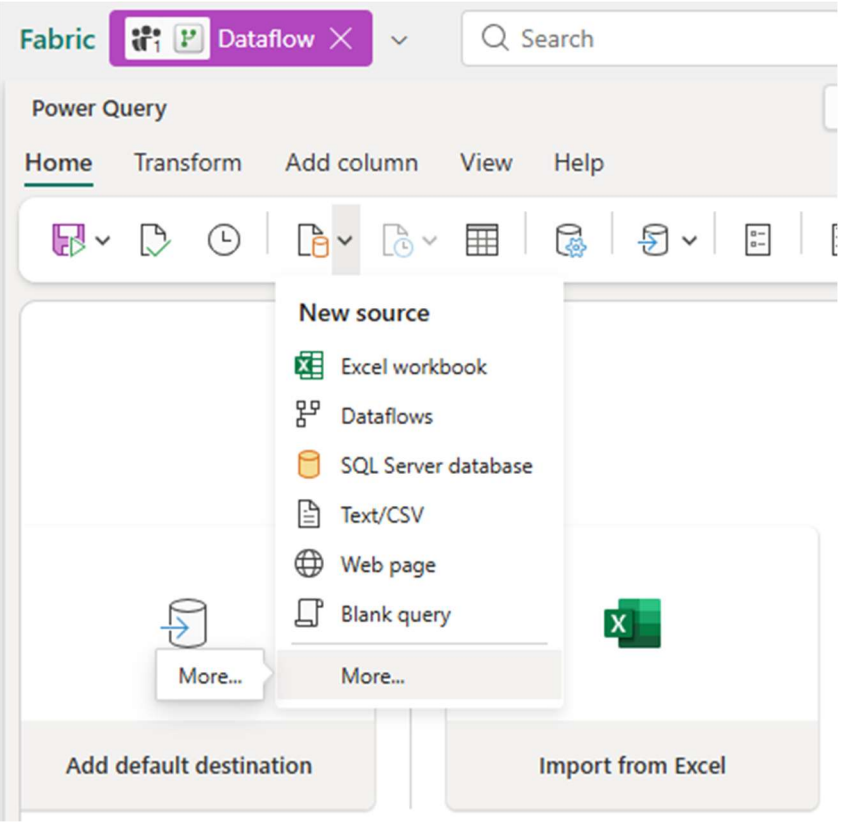




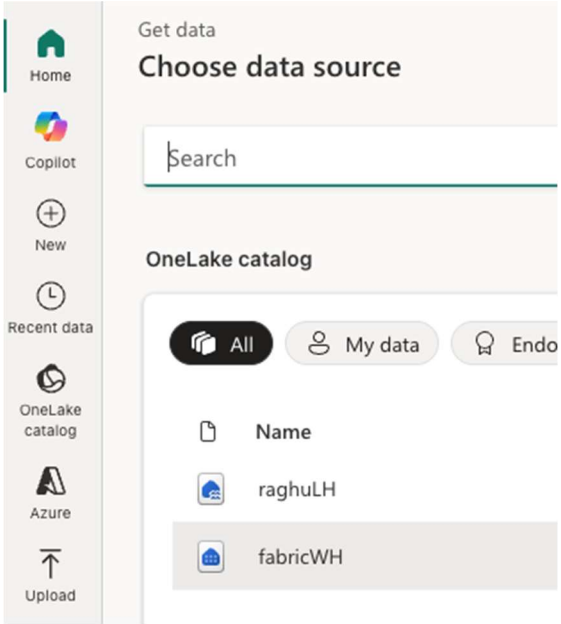
- No data copied into Fabric, but provides virtual access as a layer to perform transformations



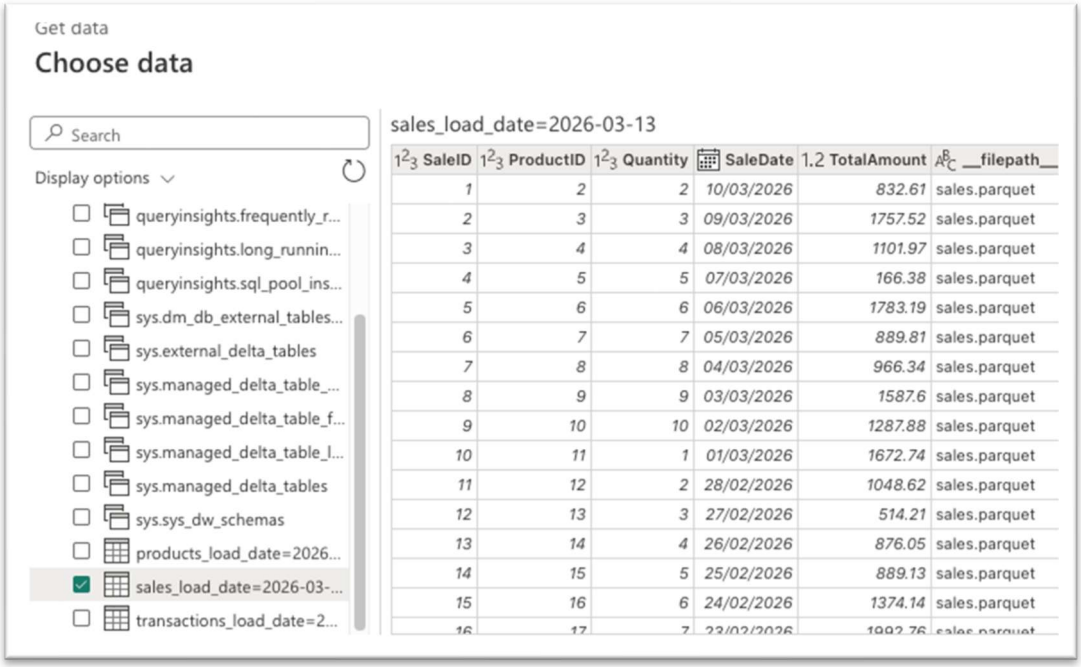
5. Data Transformation (Dataflow Gen2)



The shortcut connection virtually replicates data from source in oneLake catalog.

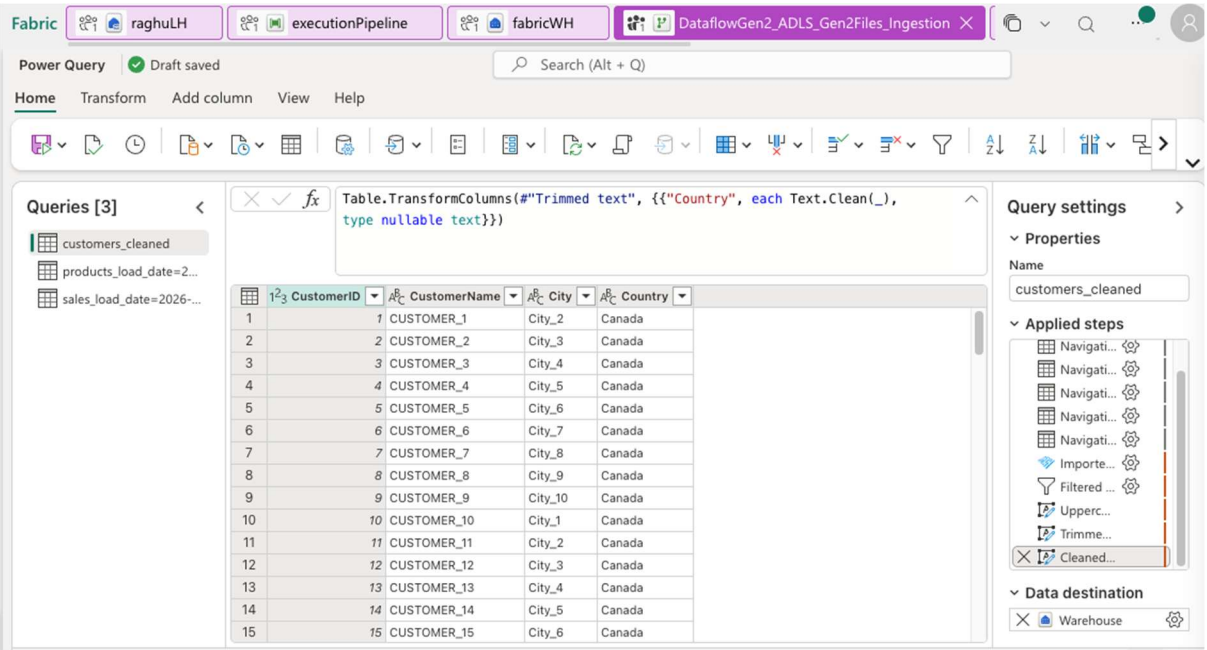


From the LakeHouse source, files are available to pick from the below delta tables.



SaleID	ProductID	Quantity	SaleDate	TotalAmount	__filepath__
1	2	2	10/03/2026	832.61	sales.parquet
2	3	3	09/03/2026	1757.52	sales.parquet
3	4	4	08/03/2026	1101.97	sales.parquet
4	5	5	07/03/2026	166.38	sales.parquet
5	6	6	06/03/2026	1783.19	sales.parquet
6	7	7	05/03/2026	889.81	sales.parquet
7	8	8	04/03/2026	966.34	sales.parquet
8	9	9	03/03/2026	1587.6	sales.parquet
9	10	10	02/03/2026	1287.88	sales.parquet
10	11	1	01/03/2026	1672.74	sales.parquet
11	12	2	28/02/2026	1048.62	sales.parquet
12	13	3	27/02/2026	514.21	sales.parquet
13	14	4	26/02/2026	876.05	sales.parquet
14	15	5	25/02/2026	889.13	sales.parquet
15	16	6	24/02/2026	1374.14	sales.parquet
16	17	7	23/02/2026	1992.76	sales.parquet

Transformations Applied:

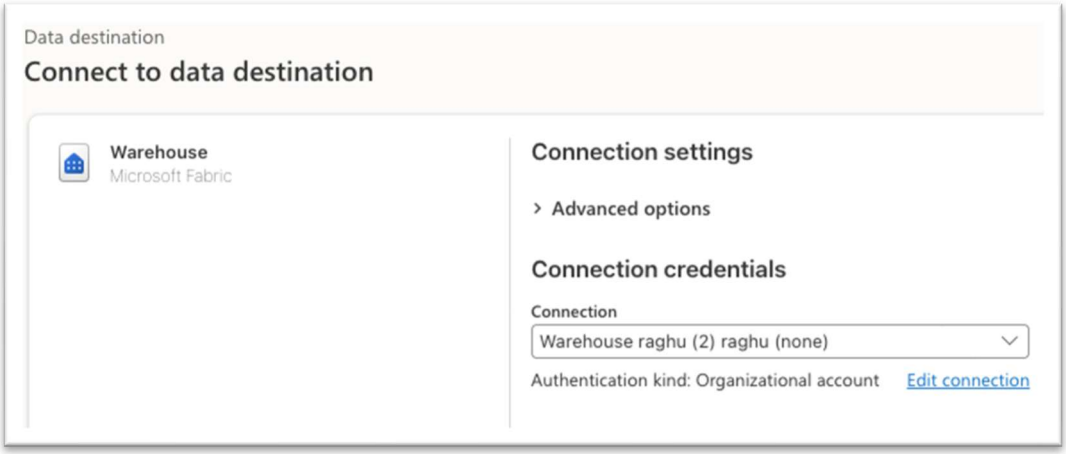


CustomerID	CustomerName	City	Country
1	CUSTOMER_1	City_2	Canada
2	CUSTOMER_2	City_3	Canada
3	CUSTOMER_3	City_4	Canada
4	CUSTOMER_4	City_5	Canada
5	CUSTOMER_5	City_6	Canada
6	CUSTOMER_6	City_7	Canada
7	CUSTOMER_7	City_8	Canada
8	CUSTOMER_8	City_9	Canada
9	CUSTOMER_9	City_10	Canada
10	CUSTOMER_10	City_1	Canada
11	CUSTOMER_11	City_2	Canada
12	CUSTOMER_12	City_3	Canada
13	CUSTOMER_13	City_4	Canada
14	CUSTOMER_14	City_5	Canada
15	CUSTOMER_15	City_6	Canada

- Data type casting
- Null handling
- Column standardization
- Filtering invalid records
- Deduplication

Destination:

- Fabric Warehouse tables



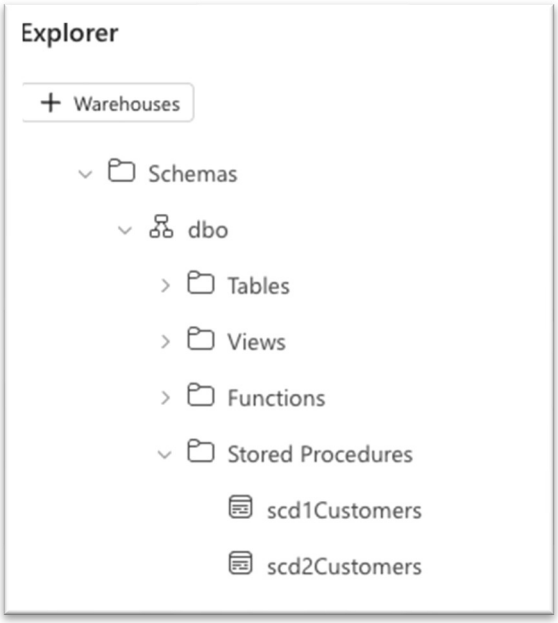
Data Warehouse Design

- Centralized structured storage
- Tables created under dbo schema

Example Tables:

- sales_load_date
- transactions_load_date

SCD Implementation: The below stored procedures are created in warehouse query editor and stored/saved in the warehouse.



SCD Type 1

- Overwrites existing records
- Used for non-historical attributes

```sql
CREATE OR ALTER PROC [dbo].[scd1Customers]
AS
BEGIN
  IF NOT EXISTS (
     SELECT 1
     FROM INFORMATION_SCHEMA.TABLES
     WHERE TABLE_SCHEMA = 'dbo'
       AND TABLE_NAME = 'dim_customers'
  )
  BEGIN
     CREATE TABLE dbo.dim_customers
     (
        CustomerID   INT NOT NULL,
        CustomerName VARCHAR(200),
        City         VARCHAR(100),
        Country      VARCHAR(100),
        HashValue    VARCHAR(64),
        CreatedBy    VARCHAR(100),
        UpdatedBy    VARCHAR(100),
        CreatedDate  DATETIME2(6),
        UpdatedDate  DATETIME2(6)
     );
  END;
  MERGE dbo.dim_customers AS tgt
  USING
  (
     SELECT
        CustomerID,
        CustomerName,
        City,
        Country,
        CONVERT(
           VARCHAR(64),
           HASHBYTES(
```

```
            'SHA2_256',
            CONCAT(
                ISNULL(CAST(CustomerName AS VARCHAR(200)), ''),
                '|',
                ISNULL(CAST(City AS VARCHAR(100)), ''),
                '|',
                ISNULL(CAST(Country AS VARCHAR(100)), '')
            )
        ),
        2
    ) AS HashValue
  FROM dbo.customers_cleaned
) AS src
  ON tgt.CustomerID = src.CustomerID
WHEN MATCHED
   AND ISNULL(tgt.HashValue, '') <> ISNULL(src.HashValue, '')
THEN
  UPDATE SET
    tgt.CustomerName = src.CustomerName,
    tgt.City         = src.City,
    tgt.Country      = src.Country,
    tgt.HashValue    = src.HashValue,
    tgt.UpdatedBy    = 'FabricPipeline',
    tgt.UpdatedDate  = SYSUTCDATETIME()

WHEN NOT MATCHED BY TARGET
THEN
  INSERT
  (
    CustomerID,
    CustomerName,
    City,
    Country,
    HashValue,
```

```
        CreatedBy,
        UpdatedBy,
        CreatedDate,
        UpdatedDate
    )
    VALUES
    (
        src.CustomerID,
        src.CustomerName,
        src.City,
        src.Country,
        src.HashValue,
        'FabricPipeline',
        'FabricPipeline',
        SYSUTCDATETIME(),
        SYSUTCDATETIME()
    );
END
```

SCD Type 2

- Maintains historical changes
- Updates for existing records on matching conditions; of primary key, hashvalue, and iscurrent flag.
- Inserts for new records where no matching business key and hashvalue.
- Uses:
    - Effective dates, StartDate and EndDate
    - Active flags

```
CREATE OR ALTER PROCEDURE dbo.scd2Customers
AS
BEGIN

  IF NOT EXISTS
  (
```

```
    SELECT 1
    FROM INFORMATION_SCHEMA.TABLES
    WHERE TABLE_SCHEMA = 'dbo'
      AND TABLE_NAME = 'dim_customersSCD2'
)
BEGIN
    CREATE TABLE dbo.dim_customersSCD2
    (
        CustomerID    INT NOT NULL,
        CustomerName  VARCHAR(200),
        City          VARCHAR(100),
        Country       VARCHAR(100),
        HashValue     VARCHAR(64),
        CreatedBy     VARCHAR(100),
        UpdatedBy     VARCHAR(100),
        StartDate DATETIME2(6),
        EndDate   DATETIME2(6),
        IsCurrent     BIT
    );
END;

UPDATE tgt
   SET tgt.EndDate = SYSUTCDATETIME(),
       tgt.IsCurrent   = 0,
       tgt.UpdatedBy   = 'FabricPipeline'
FROM dbo.dim_customersSCD2 tgt
INNER JOIN
(
    SELECT
        src.CustomerID,
        CONVERT
        (
            VARCHAR(64),
            HASHBYTES
```

```
            (
                'SHA2_256',
                CONCAT
                (
                    ISNULL(CAST(src.CustomerName AS VARCHAR(200)), ''),
                    '|',
                    ISNULL(CAST(src.City AS VARCHAR(100)), ''),
                    '|',
                    ISNULL(CAST(src.Country AS VARCHAR(100)), '')
                )
            ),
            2
        ) AS HashValue
    FROM dbo.customers_cleaned src
) src
    ON tgt.CustomerID = src.CustomerID
   AND tgt.IsCurrent = 1
WHERE ISNULL(tgt.HashValue, '') <> ISNULL(src.HashValue, '');

INSERT INTO dbo.dim_customersSCD2
(
    CustomerID,
    CustomerName,
    City,
    Country,
    HashValue,
    CreatedBy,
    UpdatedBy,
    StartDate,
    EndDate,
    IsCurrent
)
SELECT
    src.CustomerID,
```

```
    src.CustomerName,
    src.City,
    src.Country,
    src.HashValue,
    'FabricPipeline',
    'FabricPipeline',
    SYSUTCDATETIME(),
    CAST('9999-12-31 23:59:59.999999' AS DATETIME2(6)),
    1
FROM
(
    SELECT
        s.CustomerID,
        s.CustomerName,
        s.City,
        s.Country,
        CONVERT
        (
            VARCHAR(64),
            HASHBYTES
            (
                'SHA2_256',
                CONCAT
                (
                    ISNULL(CAST(s.CustomerName AS VARCHAR(200)), ''),
                    '|',
                    ISNULL(CAST(s.City AS VARCHAR(100)), ''),
                    '|',
                    ISNULL(CAST(s.Country AS VARCHAR(100)), '')
                )
            ),
            2
        ) AS HashValue
    FROM dbo.customers_cleaned s
```

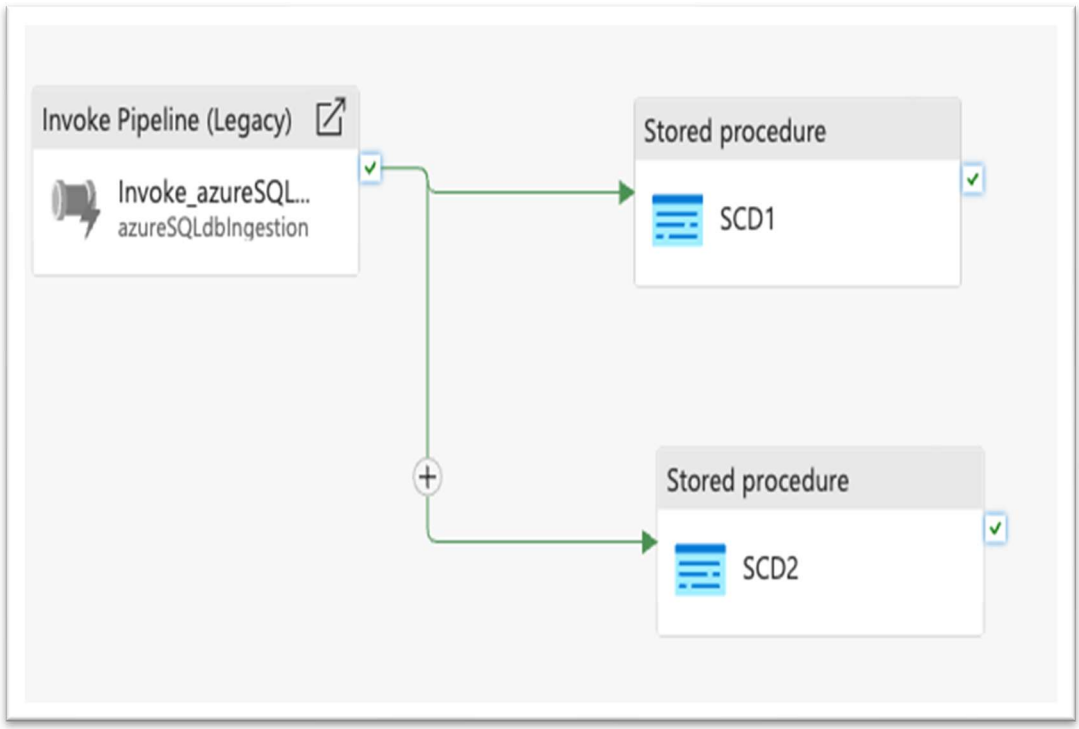
```
  ) src
  LEFT JOIN dbo.dim_customersSCD2 tgt
     ON tgt.CustomerID = src.CustomerID
    AND tgt.IsCurrent = 1
  WHERE tgt.CustomerID IS NULL
    OR ISNULL(tgt.HashValue, '') <> ISNULL(src.HashValue, '');

END;
```

6. Implementation Method

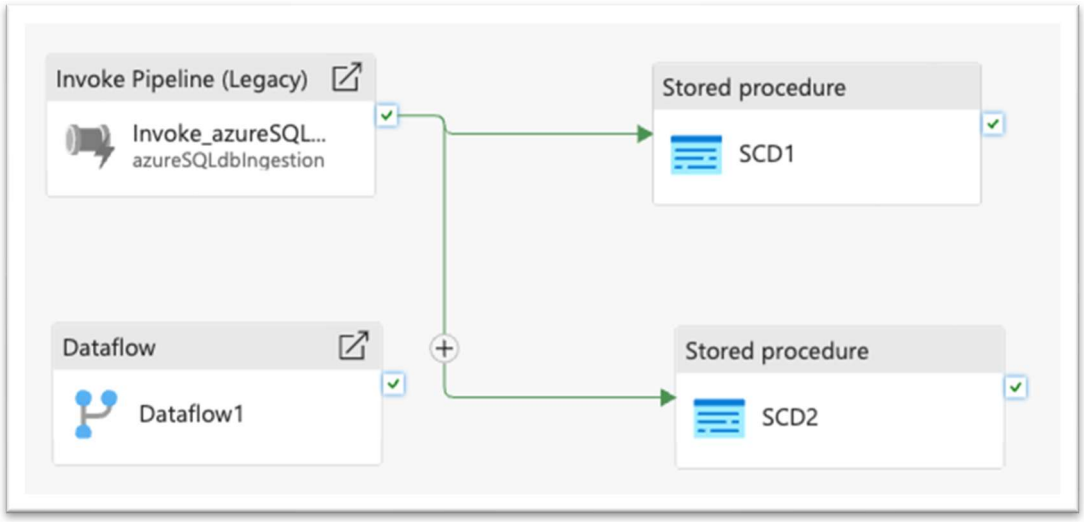
Using a Scheduled Trigger on the pipeline, pipeline can be used for incremental loads.

Implemented using:

- Child Pipeline: **Invoke Pipeline (Legacy)** of **azureSQLdbIngestion.**
- **Stored Procedures in Fabric Warehouse.**



Pipeline Orchestration



Activities:

1. Invoke Pipeline → Azure SQL DB ingestion
2. Dataflow Gen2 → ADLS data transformation
3. Stored Procedure → SCD1
4. Stored Procedure → SCD2

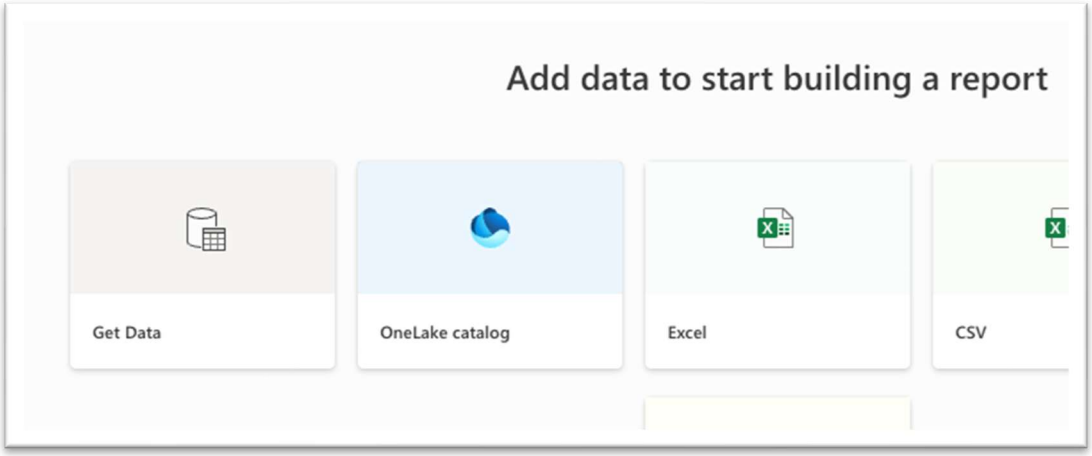
Execution Flow (Recommended)
Azure SQL Ingestion
↓
Dataflow Gen2 (ADLS)
↓
SCD1
↓
SCD2

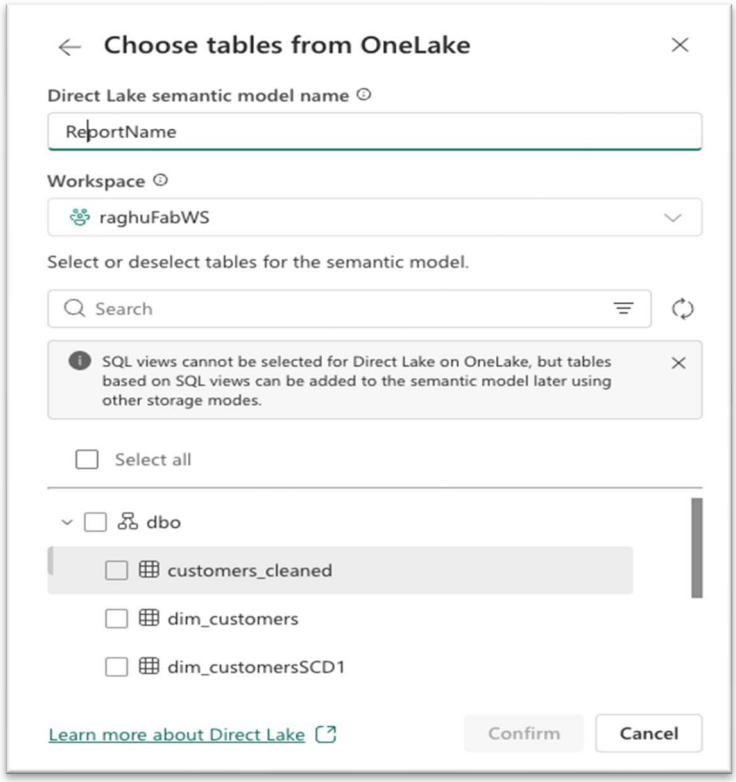
Dependency Logic

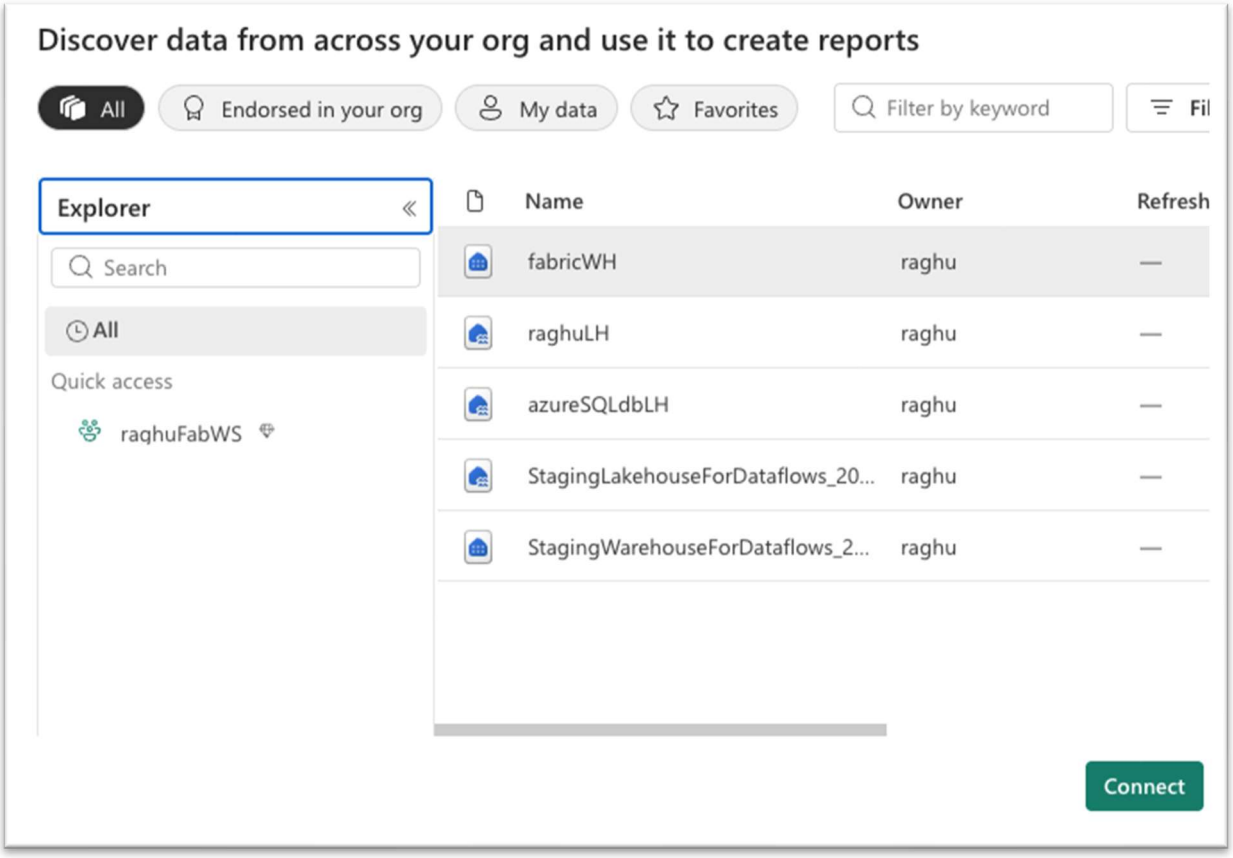
- SCD procedures executed after data load
- Ensures consistency in Warehouse

7. Semantic Model & Reporting:

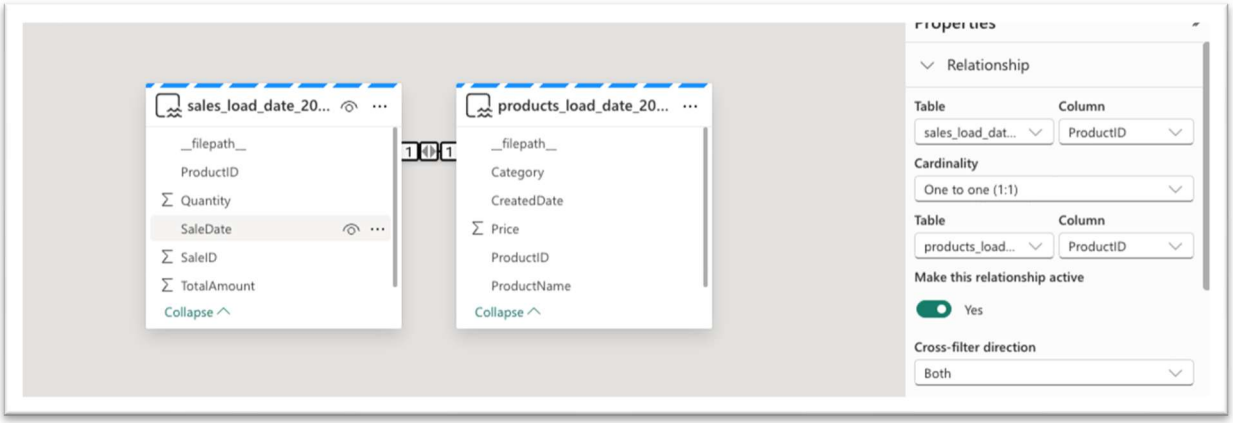
Add new item in workspace: **Semantic Model**





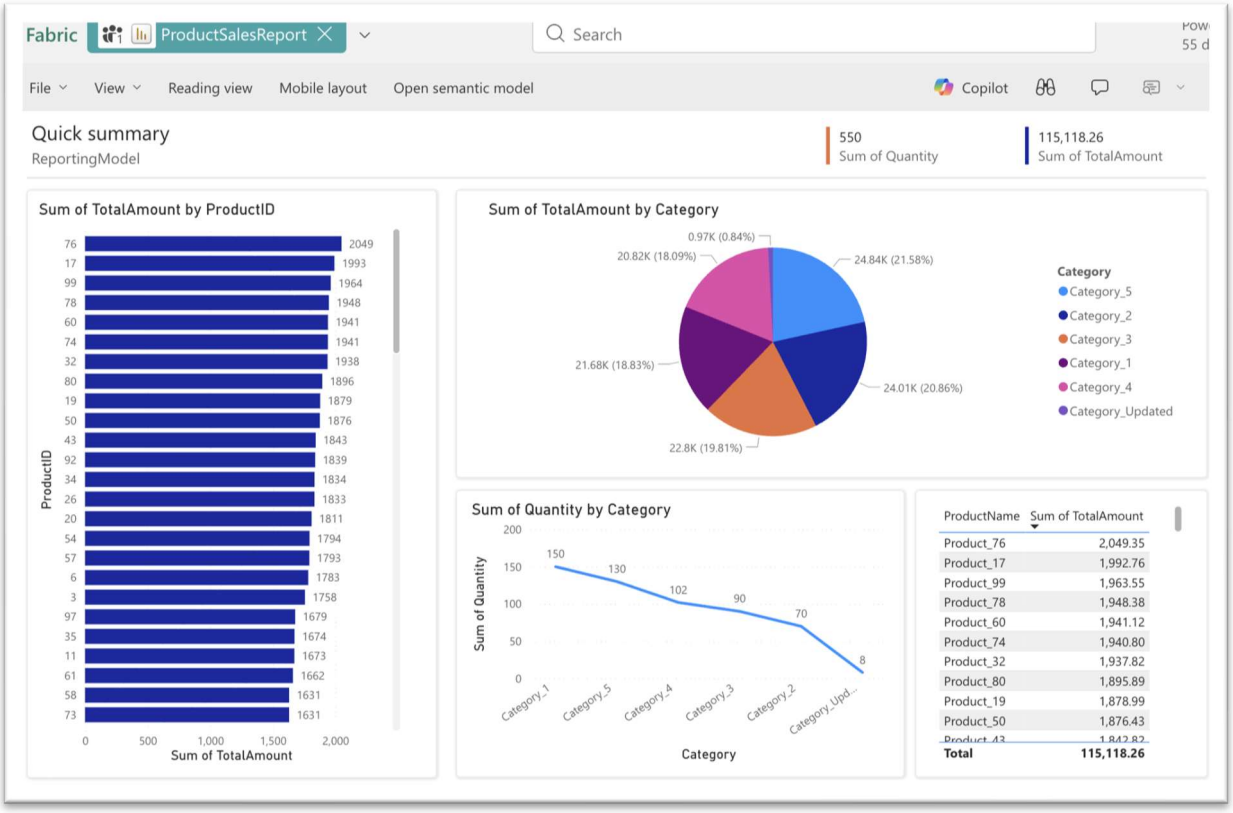


- Built on top of Warehouse tables



Features:

- Relationships defined
- Used for reporting dashboards



8. Challenges Faced

- Shortcut creation delays / visibility issues
- Handling multiple shortcut types (Tables vs Files)
- Managing pipeline dependencies
- Ensuring consistency between multiple sources

9. Learnings & Enhancements

- Multi-source ingestion
- Shortcut-based data access (no duplication)
- Dataflow Gen2 transformations
- Warehouse-based modeling
- SCD implementation
- Pipeline orchestration
- Difference between **Lakehouse Files vs Tables**
- Shortcut behavior in Fabric
- Dataflow Gen2 multi-source handling
- Pipeline orchestration best practices
- SCD implementation in Warehouse
- Implement Medallion architecture (Bronze/Silver/Gold)

- Add incremental load logic
- Use Spark notebooks for complex transformations
- Optimize Warehouse performance

10. Summary

This project demonstrates an end-to-end data engineering pipeline in Microsoft Fabric, integrating multiple data sources, applying transformations using Dataflow Gen2, and implementing SCD logic in a Warehouse, orchestrated through Fabric pipelines.